AWS MLA-C02 • BLA 04 • PART 3

Automating ML and AI Workflows with CI/CD

Deployment and Orchestration of ML and AI Workflows

03

THEORY-ONLY LEARNING SERIES

Deploy • Provision • Automate



CI/CD brings software discipline to ML

Every change to code, data, or prompts should flow through the same path.

Source

Track code, pipeline definitions, and prompts in Git.

- GitHub via CodeConnections
- CodeCommit for existing users
- Commits trigger the pipeline

Build and test

AWS CodeBuild runs tests and builds images.

- buildspec.yml defines steps
- Unit tests and data checks
- Push images to Amazon ECR

Release

AWS CodePipeline links stages; deploy tools roll out changes.

- Manual approval gates
- CloudFormation or CodeDeploy
- Dev, staging, production



SageMaker Pipelines automate the ML workflow

Each run is tracked, repeatable, and auditable.

1 Process
Prepare and validate the data

›

2 Train
Run the configured training job

›

3 Evaluate
Compute quality metrics on test data

›

4 Condition
Compare metrics to a threshold

›

5 Register
Add a version to the Model Registry

Training jobs, processing jobs, and inference jobs are configured as pipeline steps, so the whole workflow reruns the same way every time.



Version everything that changes behavior

Repeatability and audits depend on knowing exactly what ran.

Model Registry

Model package groups hold every version.

- Approval status gates deploys
- Metrics and lineage
- Approval triggers deploy

MLflow on SageMaker AI

Managed MLflow tracks experiments and models.

- Log params and metrics
- Compare runs
- Register top models

Prompt Management

Amazon Bedrock stores versioned prompts.

- Variables and settings
- Compare variants
- Pin a version in prod

Agent and FM versions

Agents use versions and aliases; tuned FMs are custom models.

- Alias points to a version
- Roll back by moving alias
- Tag each tuned model

Safe rollouts need automatic rollback

SageMaker AI deployment guardrails shift traffic gradually.

Traffic shifting

- Blue/green: new fleet beside the old one
- Canary: a small share of traffic first
- Linear: shift traffic in equal steps
- Rolling: update capacity in batches

Rollback

- CloudWatch alarms watch each step
- Baking period before the next shift
- An alarm returns traffic to the old fleet
- Keep the previous version approved



Automated tests protect ML and AI quality

Pipelines should fail fast when quality drops.

Traditional ML tests

Check data, code, and model metrics before deploy.

- Schema and data quality
- Unit and integration tests
- Metrics vs baseline

Prompt and FM tests

Run a fixed test set on every prompt or model change.

- Golden question sets
- Bedrock model evaluation
- LLM-as-a-judge checks

Agent tests

Test complete tasks, not only single responses.

- Correct tool selection
- Multi-step scenarios
- Guardrail and safety



Models and knowledge must stay current

Automated triggers keep retraining and refresh on schedule.

Retraining triggers

Start a pipeline when conditions change.

- EventBridge schedule
- Model Monitor drift alarm
- New labeled data in S3

Retraining pipeline

Rerun the same pipeline with new data.

- Same evaluation as before
- Register a new version
- Approve before deploying

RAG refresh

Sync knowledge bases when documents change.

- Scheduled ingestion jobs
- S3 events start a sync
- Retest retrieval quality



An MLOps automation checklist

Automation makes every release repeatable and reversible.

1. Store code, pipelines, and prompts in version control.
2. Build, test, and deploy through CodePipeline stages.
3. Register every model, prompt, and agent version.
4. Gate releases on automated quality tests.
5. Roll out gradually with alarm-based rollback.
6. Automate retraining and knowledge base refresh.

